

F1 Team Radio NLP Project - Viva Q&A Guide

Hochschule Aalen - M.Sc. Machine Learning & Data Analytics

Student: Shashwat Singh

1. Did you use paid external API keys (like OpenAI, Claude, or Cohere) for your RAG system?

No. The entire RAG pipeline runs locally and open-source. For semantic search and embedding calculations, we load the pre-trained 'all-MiniLM-L6-v2' model from Hugging Face via the 'sentence-transformers' library. The strategist answer generation is simulated locally based on matching rows retrieved from our F1 databases, meaning no paid API subscriptions or internet connections are required.

2. Is the code actually scraping the web and generating the CSV?

Yes. The code in the BeautifulSoup Crawler (Cell 6 / Cell index 5) uses `requests.get()` to fetch HTML transcripts from `transcripts.recursiveprojects.cloud`, parses the text nodes with BeautifulSoup, and writes the structured corpus to a CSV file on disk. In the notebook, 'SCRAPE_FROM_WEB = False' is set by default to avoid blocking notebook execution (as crawling 18,574 messages takes 10+ minutes). Flipping it to 'True' triggers a live crawler demo of 1 session and 3 drivers, overwriting the CSV file on the spot.

3. How do you identify which cell you are running inside VS Code?

Each code cell has a formatted title comment on the very first line (e.g. '# -- 1.2 BeautifulSoup Crawler'). When a cell is selected, VS Code's bottom status bar shows the absolute position (e.g., 'Cell 6 of 35' for the crawler cell). On the left of the cell, the run execution count indicator (e.g. '[' or '[5]') also shows the run order.

4. Why did you build a custom Sentiment Lexicon instead of using standard libraries like VADER?

Standard sentiment analyzers are trained on generic corpora like social media or product reviews and fail on racing jargon. In VADER, F1 commands like 'box box' (pit stop) or 'copy' (understood) are marked as neutral. 'P1' (leading the race) is ignored. Our custom lexicon maps F1 keywords directly to emotional weights - such as 'p1' (+2.0) and 'wtf' (-2.0) - to correctly reflect driver stress levels.

5. Why did the classical Linear Support Vector Machine (SVM) outperform neural networks in classification?

Our team radio messages are very short (averaging ~6 words per message). Recurrent sequence models (like RNNs and LSTMs) need long-range word dependencies to exploit their memory architecture. On short, telegraphic messages represented as sparse TF-IDF vectors, the Linear SVM finds the optimal high-margin separating hyperplane much more effectively and reaches 93.81% accuracy.

6. Explain Zipf's Law and how it applies to your F1 corpus.

Zipf's Law states that the frequency of any word in a corpus is inversely proportional to its rank. On our F1 dataset, a log-log plot of word frequency versus frequency rank displays a straight negative slope, verifying that Zipfian distribution holds. This proves that a small set of F1 core terms (e.g. 'box', 'tyre', 'gap') dominates the communication.

F1 Team Radio NLP Project - Viva Q&A Guide

Hochschule Aalen - M.Sc. Machine Learning & Data Analytics

Student: Shashwat Singh

7. How does the TF-IDF search engine work in your code?

The search engine matches user queries to transcripts using two parameters: Term Frequency (TF) - how often a word appears in a message, and Inverse Document Frequency (IDF) - the rarity of that word across the entire corpus. By computing the Cosine Similarity between the query vector and document vectors, we rank and retrieve the top matching messages.

8. How does spaCy's default NER differ from your custom NER model?

Default spaCy NER models are trained on general news articles and fail on racing terms. For example, they label constructor teams like 'Red Bull' as a PRODUCT instead of a TEAM, and miss driver names or tire compounds. We fine-tuned a blank spaCy model on 30 manually annotated F1 sentences over 30 epochs to successfully recognize DRIVER, TEAM, TIRE_COMPOUND, and STRATEGY.
